

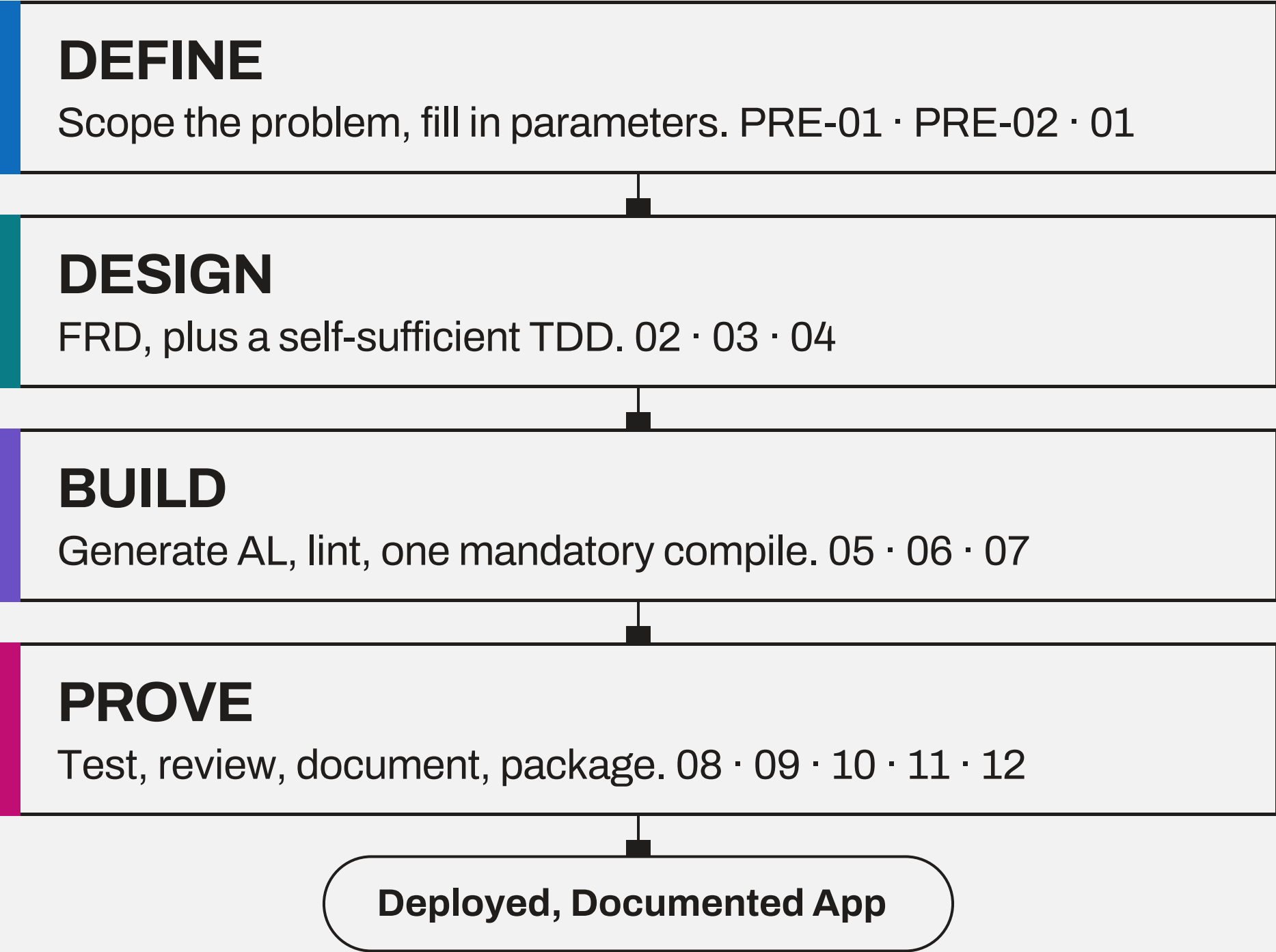


D365 BC Agentic Development Framework

**Simple Enough for Functional Consultants.
Robust Enough for Pro Developers.**

Created by AJ Ansari, Microsoft MVP

Four phases. Twelve steps. A gate on every one.



ALL ALONG — continuous discipline runs underneath every one of those four phases.

Every shape means one thing

Start / end

Start or end point of a flow

Action

An action, step, or process

Doc

Input or output — data, a document

Gate

An exit gate — met before the next step starts

Choice?

A decision point

All along

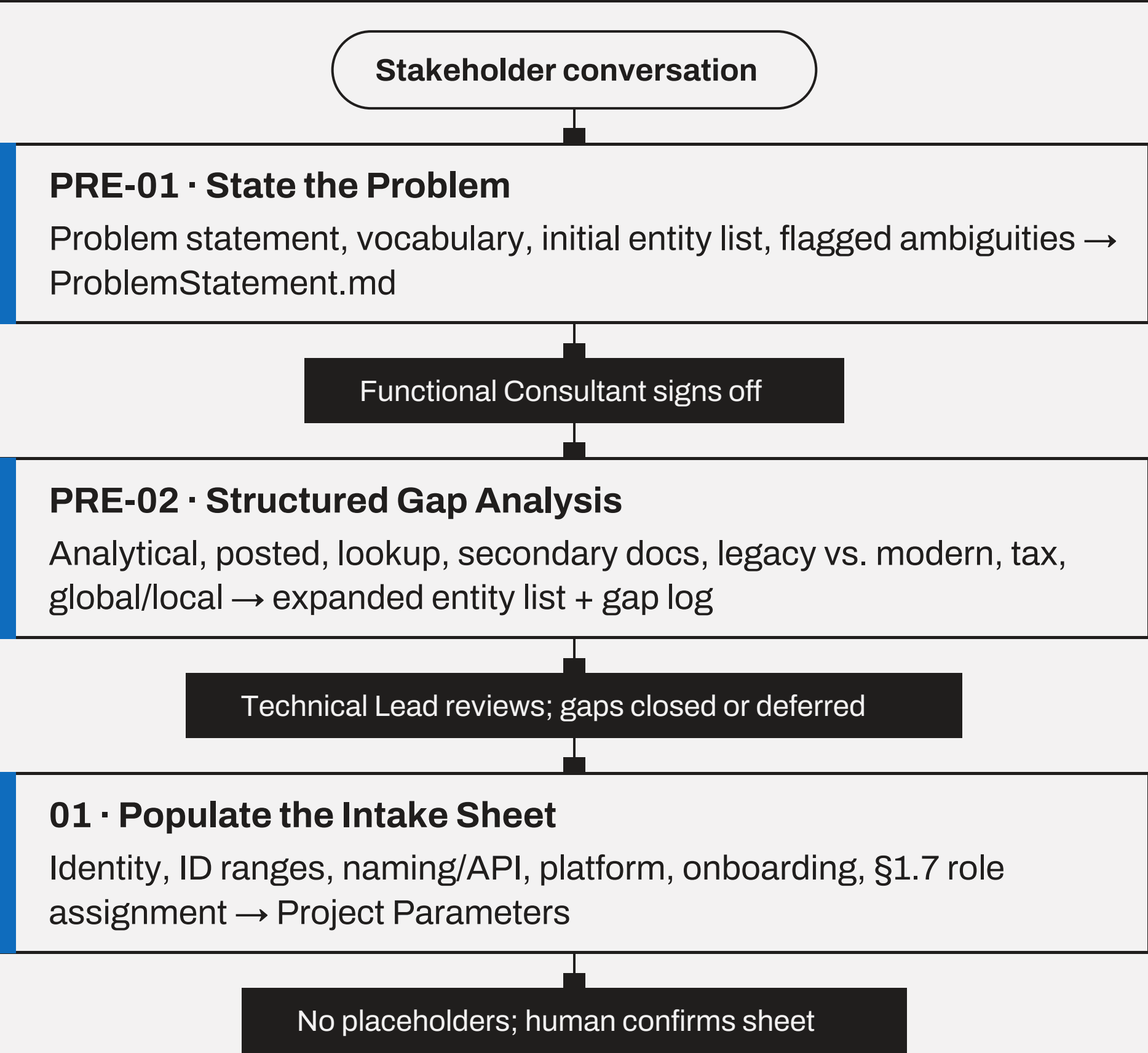
A cross-cutting concern, running underneath

Main
Writes the code

Light
Pre-flight only

Reasoning
Drafts, reviews

DEFINE — ask, don't infer



DESIGN — what, then how

02 · Craft the FRD **REASONING**

Business language — what and why, not how. Validate against DEFINE → FRD.md

Dev Manager sign-off; every entity accounted for

03 · Craft the TDD **REASONING**

System identity, module grouping, batch plan, per-object spec, permission sets — must be self-sufficient → TDD.md

Technical Lead sign-off; self-sufficiency check

04 · Sanity Check **REASONING**

Can BC actually do this? Does the TDD implement the FRD? → SanityCheck.md

0 blocking issues

The Main role integrates every draft and carries it to the human. The Reasoning role never owns sign-off.

BUILD — exactly one compile

05 · Plan the Code

Batch order, scaffold, AL MCP Server + BCQuality snapshot, pre-flight checks written

Batch order agreed; scaffold complete, not compiled

06 · Code Generation repeats per batch

Main approves and generates, then runs pre-generation pre-flight. **Light** runs post-generation pre-flight and permission-set coverage, and reports back.

↻ loop until every planned batch exists — no compile here

Every batch pre-flight clean; coverage verified

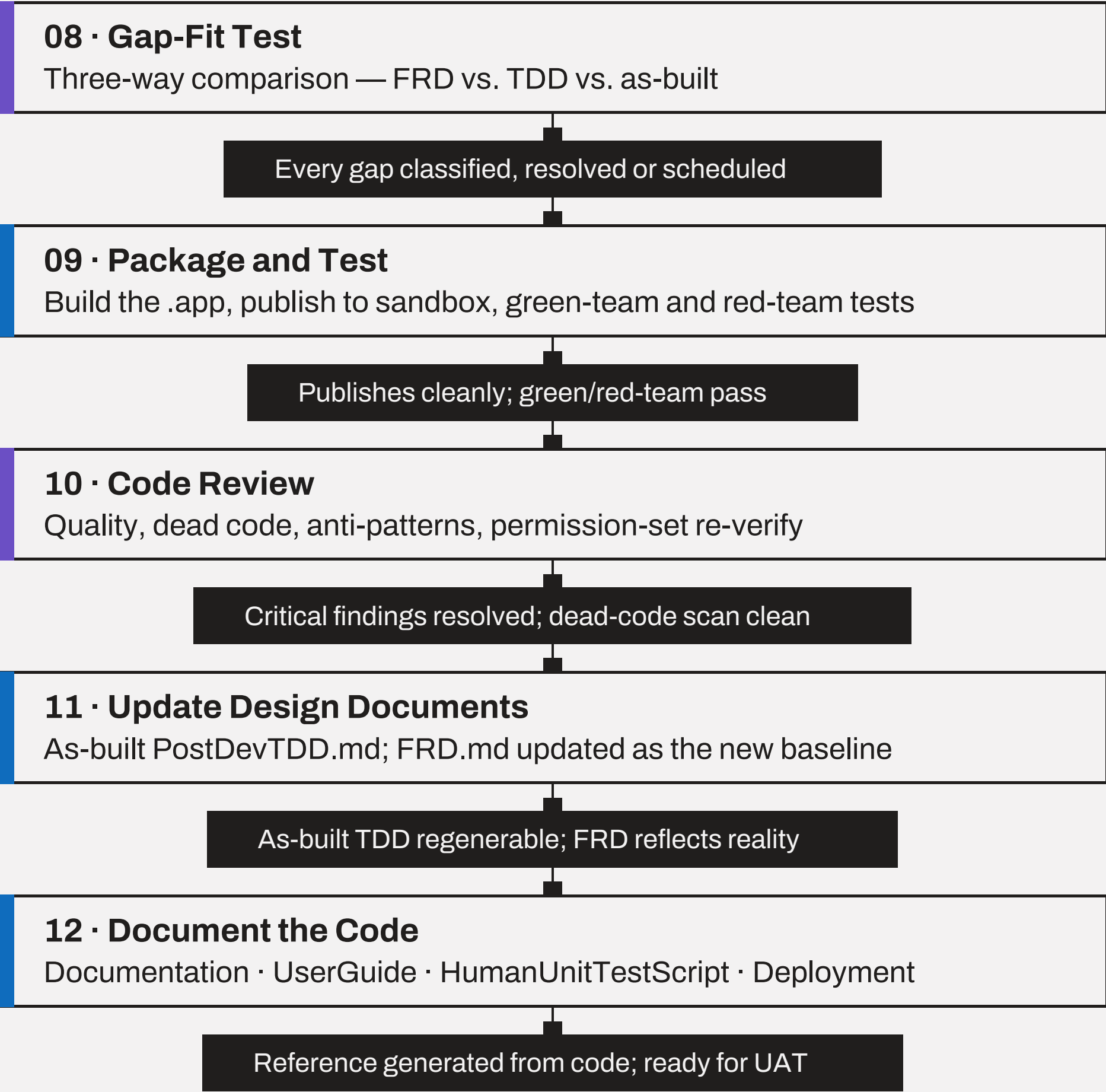
07 · Troubleshoot, Iterate

Compile the whole extension once — the one mandatory compile. **Reasoning** diagnoses root cause, **Main** fixes and re-compiles.

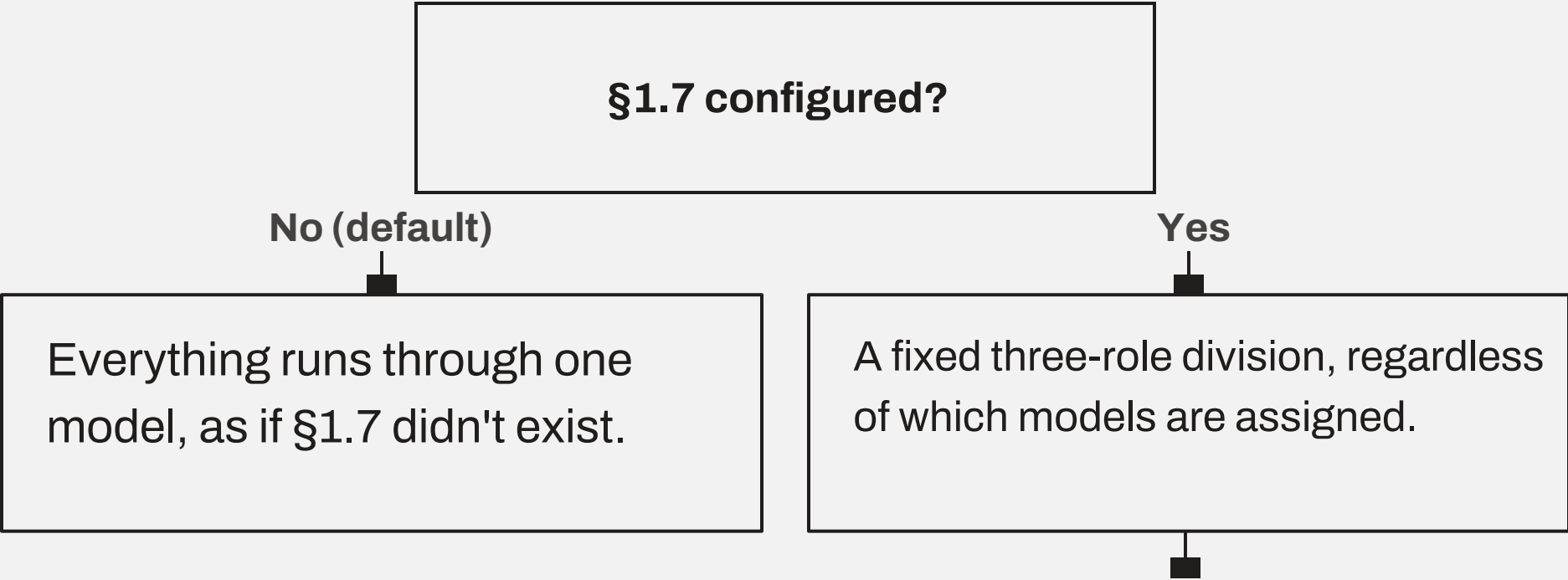
↻ repeat until clean

0 errors / 0 warnings

PROVE — five steps to UAT



Model & Effort Assignment (§1.7)



MAIN ROLE (Primary Agent; Run model like Claude Sonnet)
All code generation and every code edit, including applying what Light and Reasoning report. Owns ChangeLog, Object Register, ProjectMemory, TestingFeedback.md.

LIGHT ROLE (Sub-Agent; Run model like Claude Haiku)
The post-generation pre-flight pass only — props, Rec.-qualification, dead code, indentation, coverage, symbol verification. Reports; never edits.

REASONING ROLE (Sub-Agent; Run model like Claude Opus)
Sanity Check, Gap-Fit Test, Code Review, FRD and TDD authorship, root-cause troubleshooting. Drafts and diagnoses; never edits code or continuity docs.

ALL ALONG — every phase, every step

Document Keep every required doc current, not retroactively.	Track Changes Every FRD/TDD deviation logged before the next batch.
Retain Explanations Who decided, and why — not just the outcome.	Testing Feedback Log Verbatim findings, triaged explicitly, cross-referenced.
Project Memory docs/ProjectMemory.md — an anchor, not a narrative.	Packaging & Versioning Never delete a package; propose bumps, don't apply silently.
AL MCP Server Bootstrap once; prefer its tools over ad hoc terminal use.	BCQuality Snapshot Fetch once; refresh only on explicit request.

**Simple Enough for Functional
Consultants.
Robust Enough for Pro
Developers.**

Created by AJ Ansari, Microsoft MVP